

Desenvolvimento de Aplicações

Relatório do Projeto de Grupo

Nº Grupo: **Chipher**

Elementos do Grupo:

Nº de Aluno:	2025191285	Nome:	Alexandre Santos
Nº de Aluno:	2025182962	Nome:	Cátia Leocádio
Nº de Aluno:	2025188640	Nome:	Ivan Monteiro

Repositório GitHub

Coloque aqui o link do repositório GitHub criado e utilizado para o projeto. Caso não tenha adicionado os docentes como membros do projeto no GitHub, certifiquem-se de que o repositório está público de forma a que possa ser analisado pelos docentes

<https://github.com/AlexandreSousaSantos/PI1-3-Cipher.git>

[OPCIONAL] Extras

Enunciar extras desenvolvidos no projeto não requeridos na especificação. Caso ache conveniente, pode incluir uma breve explicação sobre os mesmos:

[OPCIONAL] Instruções especiais para correr o projeto:

A preencher no caso de o comportamento do projeto não ser o esperado e caso seja necessário fazer alguns ajustes para o projeto correr:

A base de dados é gerada automaticamente pelo Entity Framework (Code-First) na primeira execução.

Caso ocorra erro de ligação, verificar a string de ligação no ficheiro App.config e garantir que a base de dados está a correr. O projeto é compilado e executado em Visual Studio 2022 ou superior com o .NET Framework instalado.

[OPCIONAL] Observações Gerais

Caso ache conveniente, pode incluir algumas observações gerais sobre a implementação do projeto, colaboração dos elementos do grupo e/ou funcionamento do grupo de trabalho:

O grupo trabalhou de maneira coordenada onde foi utilizado um repositório Git para fazer o controlo de versões e estar tudo em um só local mas tudo organizado, dividindo os formulários e controladores de forma a garantir uma boa implementação do padrão MVC.

Requisitos Funcionais implementados

Na coluna à direita da implementação, podem colocar observações sobre cada funcionalidade, identificação do que foi feito ou do que falta fazer no trabalho desenvolvido para a cada funcionalidade

F1	LOGIN	Autenticação do utilizador. Password encriptada na base de dados.
	Completo	Foi desenvolvido a interface gráfica completa no formulário FormLogin com campos para introdução do utilizador (TButilizador) e da palavra-passe (TBpassword). Aplicámos uma validação local inicial através do evento do botão btentrar_Click utilizando .Trim(), o que impede que o utilizador insira campos vazios ou preenchidos apenas com espaços, e mostra uma mensagem de aviso ao utilizador. Integramos o formulário com a arquitetura do projeto através da chamada do controlador UtilizadorController.Autenticar(login, password, out mensagem), fazendo assim a separação de conceitos do padrão MVC. Configurámos o tratamento do fluxo pós-autenticação , por outras palavras, se os dados estiverem corretos, a View atribui DialogResult.OK e é fechada para dar lugar ao formulário principal e se falhar, limpa o campo de palavra-passe e devolve o foco ao utilizador automaticamente, ou seja, isto faz com que o programa não falhe durante a sua execução. Para que a palavra-passe não fosse mostrada, colocamos o PasswordChar nas propriedades do Text Box da palavra-passe como *. O que isto faz é que não seja mostrada quando estamos a colocar a palavra-passe no ecrã.
F2	FORM PRINCIPAL	Mostra compras em aberto e facilita o acesso ao formulário do modo compra. Apresenta um menu de acesso às restantes funcionalidades.
	Completo	O FormPrincipal carrega automaticamente todas as compras em aberto (FechadaPorId == null) numa grelha (dgvComprasAberto) ao iniciar, exibindo o ID, nome da compra, data de criação e o utilizador criador. O formulário disponibiliza botões de acesso a todas as funcionalidades da aplicação (Tipos de Artigo, Artigos, Planeamento, Estatísticas, Orçamento, Utilizadores, Exportar CSV e Sair). O botão Modo Compra abre o FormModoCompra para a compra selecionada na grelha, passando o respetivo ID.

Desenvolvimento de Aplicações

Relatório do Projeto de Grupo

Nº Grupo:

Chipher

F3	GESTÃO TIPOS ARTIGOS	CRUD de Tipos de Artigos
	Completo	O FormTiposArtigo implementa o CRUD completo de tipos de artigo. A grelha (dvgTipos) é carregada com todos os tipos através de TipoArtigoController.ListarTodos() no evento Load. O botão Novo limpa a seleção e o campo de descrição para permitir criar um novo tipo. O botão Guardar verifica se existe uma linha selecionada: se sim, chama TipoArtigoController.Atualizar(id, descricao); caso contrário chama TipoArtigoController.Criar(descricao). O botão Eliminar obtém o objeto TipoArtigo da linha selecionada e chama TipoArtigoController.Eliminar(id), atualizando a grelha de seguida. Todos os métodos estão protegidos com try/catch e mostram mensagens de erro ao utilizador.
F4	GESTÃO ARTIGOS	CRUD de Artigos
	Completo	O FormArtigos implementa o CRUD completo de artigos. No Load são carregados os tipos de artigo no ComboBox (cmbTipo) e os artigos na grelha (dvgArtigos), com projeção para mostrar o nome do tipo em vez do ID. O botão Novo limpa a seleção. O botão Guardar valida se um tipo foi selecionado, e consoante a linha selecionada na grelha chama ArtigoController.Atualizar() ou ArtigoController.Criar(). O botão Eliminar obtém o ID da linha selecionada e chama ArtigoController.Eliminar(id). Toda a lógica está envolvida em try/catch."
F5	GESTÃO ORÇAMENTOS	CRUD de Orçamentos
	Completo	O FormOrçamento implementa o CRUD completo de orçamentos mensais. O método ValidarDados() valida o formato MM/yyyy e o valor máximo. O botão Adicionar chama OrcamentoController.CriarOuAtualizar() com o mês, valor e ID do utilizador da sessão. O botão Eliminar obtém o mês da linha selecionada e chama OrcamentoController.Eliminar(mes). O botão Atualizar recarrega o orçamento do mês indicado e aplica o novo valor. A grelha (DGV_Historico_de_Orçamentos) mostra o histórico de orçamentos com formatação personalizada dos cabeçalhos e das datas.
F6	PLANEAMENTO COMPRAS	Lista de compras planeadas e acesso aos detalhes. Permite eliminar compras.
	Completo	O FormPlaneamentoCompra lista todas as compras com filtro por estado (Todas/Abertas/Fechadas) através de um ComboBox. A grelha mostra ID, nome, data de criação, estado (Aberta/Fechada), data de fecho e utilizador criador. O botão Nova Compra abre o FormEditarCompra e recarrega a lista ao fechar. O botão Editar verifica se a compra está fechada antes de abrir o FormEditarCompra. O botão Eliminar chama CompraController.EliminarCompra(id) para a linha selecionada. O filtro de estado atualiza a lista automaticamente ao mudar a seleção.
F7	DETALHES COMPRA PLANEADA	Criação e alteração de uma compra e respetivos itens. Apenas leitura caso esteja fechada.
	Completo	O FormEditarCompra suporta dois modos: criação (construtor sem parâmetros) e edição (construtor com compralId). Em modo de criação, os controlos de itens ficam desativados até a compra ser guardada. O botão Guardar Compra cria uma nova Compra (com CriadoPorId da sessão) ou atualiza o nome da existente. O botão Adicionar insere um novo ItemPrevisto com artigo e quantidade selecionados. O botão Remover Item elimina o item selecionado da grelha. O botão Editar Artigo permite alterar o artigo de um item já existente. Se a compra estiver fechada (FechadaPorId != null), o método BloquearFormularioLeitura() desativa todos os controlos e informa o utilizador.
F8	MODO COMPRA	Visualização de uma compra planeada em aberto para realizar a compra dos itens, adicionar itens não previstos e fechar a compra.
	Completo	O FormModoCompra recebe o ID de uma compra e constrói a grelha manualmente com colunas Id (oculto), Tipo, Artigo, Qt Prevista, Qt Adquirida, Preço Unit., Adquirido e Observações. Os itens previstos (ItemPrevisto) e não previstos (ItemNaoPrevisto) são carregados separadamente via ItemCompraController e apresentados na mesma grelha com a coluna Tipo a distingui-los. O botão Marcar Adquirido chama ItemCompraController.MarcarComoAdquirido() com a quantidade e preço selecionados, apenas para itens do tipo Previsto. O botão Adicionar cria um novo ItemNaoPrevisto com artigo, quantidade e preço introduzidos. O botão Fechar Compra pede confirmação e chama CompraController.FecharCompra(), o que valida sempre que existe um utilizador em sessão.
	ESTATÍSTICAS E APOIO À DECISÃO	Apresentação das estatísticas pedidas e possibilidade de pedido de sugestão de orçamento e de compras.

Desenvolvimento de Aplicações

Relatório do Projeto de Grupo

Nº Grupo:

Chipher

F9	Completo	O FormEstatisticas carrega quatro grelhas no evento Load, cada uma populada pelo EstatisticasController: dgvResumoMes com o resumo mensal (mês, ano, orçamento, total de compras e diferença). O dgvComprasFechadas com estatísticas das compras fechadas (total de itens, previstos, não previstos e percentagem de previstos). O dgvSugestoesOrçamento com a sugestão de orçamento para o próximo mês (média histórica, último orçamento e sugestão calculada como média dos dois). E o dgvSugestoesCompra com a sugestão de compras agrupada por semana do mês.
----	----------	--

Requisitos não-funcionais implementados

Na coluna à direita da implementação, podem colocar observações sobre cada funcionalidade, identificação do que foi feito ou do que falta fazer no trabalho desenvolvido para a cada funcionalidade

NF1	UTILIZAÇÃO DO ENTITY FRAMEWORK	Correta utilização do EntityFramework, desde geração da base de dados e classes associadas, até à utilização do Container juntamente com a tecnologia LINQ onde aplicável
	Completo	O projeto utiliza o Entity Framework 6 em modo Code-First. O iShoppingContext herda de DbContext e expõe os DbSet de todas as entidades (Utilizadores, Artigos, TiposArtigo, Compras, ItemCompra, Orcamento). O AppDbInitializer adiciona a base de dados com dados de seed na primeira execução. Em todos os controladores, os acessos à base de dados são feitos dentro de blocos using (iShoppingContext db = new iShoppingContext()), o que garante a libertação de recursos. As consultas utilizam LINQ (Where, Select, FirstOrDefault, Any, GroupBy, Average, Sum, etc.) tanto em sintaxe de método como em sintaxe de query.
NF2	UTILIZAÇÃO DE ARQUITETURA MVC	Utilização da arquitetura MVC para conferir modularização à aplicação, bem como a correta reutilização de código
	Completo	O projeto está organizado nas três camadas do padrão MVC: Models (pasta Models) com as classes de entidade geradas pelo EF (Utilizador, Artigo, TipoArtigo, Compra, ItemCompra, ItemPrevisto, ItemNaoPrevisto, Orcamento, Sessao, iShoppingContext). Os Controllers (pasta Controller) com os controladores que encapsulam toda a lógica de negócio e acesso à base de dados (UtilizadorController, ArtigoController, TipoArtigoController, CompraController, ItemCompraController, OrcamentoController, EstatisticasController). E por último as Views (pasta Views) com os formulários Windows Forms que apenas invocam os controladores e apresentam resultados ao utilizador, sem lógica de negócio direta.
NF3	PROTEÇÕES CONTRA FALHAS	Utilização correta e frequente de código e lógica para evitar potenciais falhas e/ou crashes da aplicação, tais como verificações de nulls ou blocos try/catch
	Completo	Todos os formulários envolvem as operações críticas em blocos try/catch com MessageBox.Show() de erro ao utilizador. Os controladores validam parâmetros de entrada (IsNullOrWhiteSpace, campos obrigatórios) antes de aceder à base de dados e devolvem mensagens descritivas via parâmetro out. São feitas verificações de null em objetos retornados da BD. O FormModoCompra verifica Sessao.UtilizadorAtual != null antes de fechar a compra. O FormEditarCompra verifica _compralid > 0 antes de adicionar itens. O UtilizadorController.Eliminar() captura DbUpdateException para tratar erros de integridade referencial. Os métodos de listagem devolvem listas vazias em caso de ausência de dados.
NF4	REPOSITÓRIO GIT	Ao longo do desenvolvimento de todo o projeto foi feita uma correta utilização do GIT/GitHub por todos os estudantes do projeto, com vários commits efetuados que demonstram a evolução do projeto ao longo das semanas
	Completo	O repositório GitHub (https://github.com/AlexandreSousaSantos/PI1-3-Cipher) foi utilizado por todos os elementos do grupo ao longo dos três sprints do projeto. Os commits registam a evolução incremental do código, desde a estrutura inicial até à implementação completa de todas as funcionalidades. O projeto seguiu a metodologia Scrum com sprints definidos, tendo o repositório servido de central de integração do trabalho de todos os elementos.